

FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING

**Department of Computer Engineering
Academic Year 2025-26**

Rubrics for Lab Experiments

**Class : B.E. Computer
Engineering**

Subject Name :DC Lab

Semester : VIII

Subject Code : CSL801

Practical No:	8
Title:	Implement a simplified version of the MapReduce framework for distributed processing
Date of Performance:	18/11/25
Roll No:	9913
Name of the Student:	Mark Lopes

Evaluation:

Performance Indicator	Below average	Average	Good	Excellent	Marks
On time Submission (2)	Not submitted (0)	Submitted after deadline (1)	Early or on time submission(2)	---	
Test cases and output (4)	Incorrect output (1)	The expected output is verified only a for few test cases (2)	The expected output is Verified for all test cases but is not presentable (3)	Expected output is obtained for all test cases. Presentable and easy to follow (4)	
Coding efficiency (2)	The code is not structured at all (0)	The code is structured but not efficient (1)	The code is Structured and efficient. (2)	-	
Knowledge(2)	Basic concepts not clear (0)	Understood the basic concepts (1)	Could explain the concept with suitable example (1.5)	Could relate the theory with real world application(2)	
Total					

Aim:

Implement a simplified version of the MapReduce framework for distributed processing

Collector.java

```
package exp8;

import java.util.List;
import java.util.ArrayList;

public class Collector {
    private final List<KV> list = new ArrayList<>();
    public void emit(String k, String v) { list.add(new KV(k, v)); }
    public List<KV> get() { return list; }
}
```

JobClient.java

```
package exp8;

import java.io.*;
import java.net.*;
import java.util.*;

public class JobClient {
    public static void main(String[] args) throws Exception {
        if (args.length != 7) {
            System.out.println("Usage: java exp8.JobClient <masterHost> <masterPort> <inputPath> <mapperClass> <reducerClass> <splits> <outPath>");
            return;
        }
        String mh = args[0]; int mp = Integer.parseInt(args[1]);
        String input = args[2], mapper = args[3], reducer = args[4];
        int splits = Integer.parseInt(args[5]); String out = args[6];

        try (Socket s = new Socket(mh, mp);
            ObjectOutputStream outo = new ObjectOutputStream(s.getOutputStream());
            ObjectInputStream ino = new ObjectInputStream(s.getInputStream())) {
            Map<String, Object> data = new HashMap<>();
            data.put("inputPath", input);
            data.put("mapper", mapper);
            data.put("reducer", reducer);
            data.put("splits", splits);
            data.put("outPath", out);
        }
    }
}
```

```

// IMPORTANT: use SUBMIT instead of null
outo.writeObject(new MRRequest(MRRequest.Type.SUBMIT, data));
outo.flush();

MRResponse resp = (MRResponse) ino.readObject();
if (resp.ok) System.out.println("Job finished: " + resp.data);
else System.err.println("Job failed: " + resp.error);
    }
}
}

```

KV.java

```

package exp8;

import java.io.Serializable;

public class KV implements Serializable {
    public final String key;
    public final String value;
    public KV(String k, String v) { this.key = k; this.value = v; }
    public String toString() { return key + "\t" + value; }
}

```

Mapper.java

```

package exp8;

public interface Mapper {
    // key: split id or line number; value: input text
    void map(String key, String value, Collector collector);
}

```

Master.java

```

package exp8;

import java.io.*;
import java.net.*;
import java.util.*;
import java.util.concurrent.*;

public class Master {
    private final int port;
    private final List<String> workers = Collections.synchronizedList(new ArrayList<>());
    private final ExecutorService pool = Executors.newCachedThreadPool();
}

```

```

public Master(int port) {
    this.port = port;
}

public void start() throws Exception {
    ServerSocket ss = new ServerSocket(port);
    System.out.println("Master listening on " + port);
    // accept workers and job clients
    while (true) {
        Socket s = ss.accept();
        pool.submit(() -> handle(s));
    }
}

private void handle(Socket s) {
    try (ObjectInputStream in = new ObjectInputStream(s.getInputStream());
        ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream())) {

        MRRequest req = (MRRequest) in.readObject();

        // defensive check
        if (req == null || req.type == null) {
            out.writeObject(MRResponse.fail("Invalid request or null request type"));
            out.flush();
            return;
        }

        switch (req.type) {
            case REGISTER: {
                String addr = (String) req.data.get("address");
                if (!workers.contains(addr)) { workers.add(addr); }
                System.out.println("Worker registered: " + addr);
                out.writeObject(MRResponse.ok(Map.of("message", "registered")));
                break;
            }
            case SUBMIT: {
                // job submission data: "inputPath", "mapper", "reducer", "splits", "outPath"
                String inputPath = (String) req.data.get("inputPath");
                String mapperClass = (String) req.data.get("mapper");
                String reducerClass = (String) req.data.get("reducer");
                int splits = (Integer) req.data.get("splits");
                String outputPath = (String) req.data.get("outPath");
                System.out.println("Job received: mapper=" + mapperClass + " reducer=" +
reducerClass + " splits=" + splits);
                try {
                    List<List<String>> splitData = createSplits(inputPath, splits);
                    List<List<KV>> intermediate = assignMapTasks(splitData, mapperClass);
                    Map<String, List<String>> shuffled = shuffle(intermediate);

```

```

        Map<String, String> finalOutput = assignReduceTasks(shuffled,
reducerClass);
        // write output
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(outPath))) {
            for (var e : finalOutput.entrySet()) {
                bw.write(e.getKey() + "\t" + e.getValue());
                bw.newLine();
            }
        }
        out.writeObject(MRResponse.ok(Map.of("message", "job finished", "output",
outPath)));
    } catch (Exception ex) {
        ex.printStackTrace();
        out.writeObject(MRResponse.fail("job failed: " + ex.getMessage()));
    }
    break;
}
case MAP_TASK:
case REDUCE_TASK:
    out.writeObject(MRResponse.fail("Master doesn't perform map/reduce
directly"));
    break;
default:
    out.writeObject(MRResponse.fail("Unsupported request type"));
}
out.flush();
} catch (Exception e) {
    System.err.println("Master handler error: " + e);
} finally {
    try { s.close(); } catch (Exception ignored) {}
}
}
}

```

```

private List<List<String>> createSplits(String inputPath, int splits) throws IOException {
    List<String> lines = new ArrayList<>();
    try (BufferedReader br = new BufferedReader(new FileReader(inputPath))) {
        String l;
        while ((l = br.readLine()) != null) lines.add(l);
    }
    List<List<String>> out = new ArrayList<>();
    for (int i=0; i<splits; i++) out.add(new ArrayList<>());
    for (int i=0; i<lines.size(); i++) out.get(i % splits).add(lines.get(i));
    System.out.println("Created " + splits + " splits");
    return out;
}

```

```

private List<List<KV>> assignMapTasks(List<List<String>> splitData, String
mapperClass) throws Exception {

```

```

List<List<KV>> results = Collections.synchronizedList(new ArrayList<>());
CountDownLatch latch = new CountDownLatch(splitData.size());
for (int i=0;i<splitData.size();i++) {
    final int idx = i;
    pool.submit() -> {
        try {
            String worker = pickWorker(idx);
            List<KV> res = sendMapTask(worker, mapperClass, "split-" + idx,
splitData.get(idx));
            results.add(res);
            System.out.println("Map done for split " + idx + " on " + worker + " produced " +
res.size() + " kvs");
        } catch (Exception e) {
            throw new RuntimeException(e);
        } finally { latch.countDown(); }
    });
}
try { latch.await(); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
return results;
}

```

```

private String pickWorker(int idx) {
    if (workers.isEmpty()) throw new RuntimeException("No workers available");
    return workers.get(idx % workers.size());
}

```

```

@SuppressWarnings("unchecked")
private List<KV> sendMapTask(String workerAddr, String mapperClass, String splitId,
List<String> lines) throws Exception {
    String[] hp = workerAddr.split(":");
    try (Socket s = new Socket(hp[0], Integer.parseInt(hp[1]));
        ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream());
        ObjectInputStream in = new ObjectInputStream(s.getInputStream())) {
        Map<String, Object> data = new HashMap<>();
        data.put("mapper", mapperClass);
        data.put("splitId", splitId);
        data.put("lines", lines);
        out.writeObject(new MRRequest(MRRequest.Type.MAP_TASK, data));
        out.flush();
        MRResponse resp = (MRResponse) in.readObject();
        if (!resp.ok) throw new RuntimeException("Map failed: " + resp.error);
        return (List<KV>) resp.data.get("kvs");
    }
}

```

```

private Map<String, List<String>> shuffle(List<List<KV>> intermediate) {
    Map<String, List<String>> map = new HashMap<>();
    for (List<KV> list : intermediate) {

```

```

        for (KV kv : list) {
            map.computeIfAbsent(kv.key, k -> new ArrayList<>()).add(kv.value);
        }
    }
    System.out.println("Shuffle produced " + map.size() + " keys");
    return map;
}

```

```

private Map<String, String> assignReduceTasks(Map<String, List<String>> shuffled,
String reducerClass) throws Exception {
    Map<String, String> finalMap = new ConcurrentHashMap<>();
    List<String> keys = new ArrayList<>(shuffled.keySet());
    CountDownLatch latch = new CountDownLatch(keys.size());
    for (int i=0;i<keys.size();i++) {
        final String key = keys.get(i);
        final String worker = pickWorker(i);
        pool.submit(() -> {
            try {
                String outv = sendReduceTask(worker, reducerClass, key, shuffled.get(key));
                finalMap.put(key, outv);
                System.out.println("Reduce done for key " + key + " on " + worker);
            } catch (Exception e) {
                throw new RuntimeException(e);
            } finally { latch.countDown(); }
        });
    }
    try { latch.await(); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    return finalMap;
}

```

```

private String sendReduceTask(String workerAddr, String reducerClass, String key,
List<String> values) throws Exception {
    String[] hp = workerAddr.split(":");
    try (Socket s = new Socket(hp[0], Integer.parseInt(hp[1]));
        ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream());
        ObjectInputStream in = new ObjectInputStream(s.getInputStream())) {
        Map<String, Object> data = new HashMap<>();
        data.put("reducer", reducerClass);
        data.put("key", key);
        data.put("values", values);
        out.writeObject(new MRRequest(MRRequest.Type.REDUCE_TASK, data));
        out.flush();
        MRResponse resp = (MRResponse) in.readObject();
        if (!resp.ok) throw new RuntimeException("Reduce failed: " + resp.error);
        return (String) resp.data.get("result");
    }
}

```

```

    public static void main(String[] args) throws Exception {
        if (args.length != 1) { System.out.println("Usage: java exp8.Master <port>"); return; }
        int port = Integer.parseInt(args[0]);
        new Master(port).start();
    }
}

```

MRRequest.java

```

package exp8;

import java.io.Serializable;
import java.util.Map;

public class MRRequest implements Serializable {
    public enum Type { REGISTER, MAP_TASK, REDUCE_TASK, HEARTBEAT, SUBMIT }
    public final Type type;
    public final Map<String, Object> data;
    public MRRequest(Type t, Map<String, Object> d) { this.type = t; this.data = d; }
}

```

MRResponse.java

```

package exp8;

import java.io.Serializable;
import java.util.Map;

public class MRResponse implements Serializable {
    public final boolean ok;
    public final String error;
    public final Map<String, Object> data;
    public MRResponse(boolean ok, String err, Map<String, Object> data) {
        this.ok = ok; this.error = err; this.data = data;
    }
    public static MRResponse ok(Map<String, Object> d) { return new MRResponse(true, null, d); }
    public static MRResponse fail(String err) { return new MRResponse(false, err, null); }
}

```

Reducer.java

```

package exp8;
import java.util.List;

public interface Reducer {

```



```
// key and list of values for that key
void reduce(String key, List<String> values, Collector collector);
}
```

WordCountMapper.java

```
package exp8;

public class WordCountMapper implements Mapper {
    public void map(String key, String value, Collector collector) {
        String[] toks = value.split("\\W+");
        for (String t : toks) {
            if (t.length() > 0) collector.emit(t.toLowerCase(), "1");
        }
    }
}
```

WordCountReducer.java

```
package exp8;

import java.util.List;

public class WordCountReducer implements Reducer {
    public void reduce(String key, List<String> values, Collector collector) {
        int sum = 0;
        for (String v : values) sum += Integer.parseInt(v);
        collector.emit(key, Integer.toString(sum));
    }
}
```

Worker.java

```
package exp8;

import java.io.*;
import java.net.*;
import java.util.*;
import java.lang.reflect.*;

public class Worker {
    private final String masterHost;
    private final int masterPort;
    private final int port;

    public Worker(String masterHost, int masterPort, int port) {
```

```

        this.masterHost = masterHost;
        this.masterPort = masterPort;
        this.port = port;
    }

    public void start() throws Exception {
        register();
        ServerSocket ss = new ServerSocket(port);
        System.out.println("Worker listening on " + port);
        while (true) {
            Socket s = ss.accept();
            new Thread(() -> handle(s)).start();
        }
    }

    private void register() {
        try (Socket s = new Socket(masterHost, masterPort);
            ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream());
            ObjectInputStream in = new ObjectInputStream(s.getInputStream())) {
            String addr = InetAddress.getLocalHost().getHostAddress() + ":" + port;
            out.writeObject(new MRRequest(MRRequest.Type.REGISTER, Map.of("address",
addr)));
            out.flush();
            MRResponse r = (MRResponse) in.readObject();
            if (!r.ok) System.err.println("Register failed: " + r.error);
        } catch (Exception e) {
            System.err.println("Register error: " + e.getMessage());
        }
    }

    private void handle(Socket s) {
        try (ObjectInputStream in = new ObjectInputStream(s.getInputStream());
            ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream())) {
            MRRequest req = (MRRequest) in.readObject();
            if (req.type == MRRequest.Type.MAP_TASK) {
                String mapperClass = (String) req.data.get("mapper");
                String splitId = (String) req.data.get("splitId");
                @SuppressWarnings("unchecked")
                List<String> lines = (List<String>) req.data.get("lines");
                List<KV> kvs = doMap(mapperClass, splitId, lines);
                out.writeObject(MRResponse.ok(Map.of("kvs", kvs)));
            } else if (req.type == MRRequest.Type.REDUCE_TASK) {
                String reducerClass = (String) req.data.get("reducer");
                String key = (String) req.data.get("key");
                @SuppressWarnings("unchecked")
                List<String> values = (List<String>) req.data.get("values");
                String res = doReduce(reducerClass, key, values);
                out.writeObject(MRResponse.ok(Map.of("result", res)));
            }
        }
    }

```

```

        } else {
            out.writeObject(MRResponse.fail("Unsupported at worker"));
        }
        out.flush();
    } catch (Exception e) {
        System.err.println("Worker handler error: " + e);
    } finally {
        try { s.close(); } catch (Exception ignored) {}
    }
}

private List<KV> doMap(String mapperClass, String splitId, List<String> lines) throws
Exception {
    Mapper mapper = (Mapper)
Class.forName(mapperClass).getDeclaredConstructor().newInstance();
    Collector c = new Collector();
    int lineno = 0;
    for (String line : lines) {
        mapper.map(splitId + ":" + lineno, line, c);
        lineno++;
    }
    return c.get();
}

private String doReduce(String reducerClass, String key, List<String> values) throws
Exception {
    Reducer reducer = (Reducer)
Class.forName(reducerClass).getDeclaredConstructor().newInstance();
    Collector c = new Collector();
    reducer.reduce(key, values, c);
    // join emitted values into single string (simple)
    StringBuilder sb = new StringBuilder();
    for (KV kv : c.get()) {
        if (sb.length() > 0) sb.append(",");
        sb.append(kv.value);
    }
    return sb.toString();
}

public static void main(String[] args) throws Exception {
    if (args.length != 3) { System.out.println("Usage: java Worker <masterHost>
<masterPort> <port>"); return; }
    new Worker(args[0], Integer.parseInt(args[1]), Integer.parseInt(args[2])).start();
}
}

```

```
Welcome to Hallownest, little ghost...
```

```
Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp8
```

```
• > javac *.java
```

```
Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp8
```

```
❖ > java Master 9000
```

```
Master listening on 9000
```

```
█
```

```
Welcome to Hallownest, little ghost...
```

```
Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp8
```

```
○ > java Worker localhost 9000 10001
```

```
Worker listening on 10001
```

```
█
```

```
Welcome to Hallownest, little ghost...
```

```
Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp8
```

```
○ > java Worker localhost 9000 10002
```

```
Worker listening on 10002
```

```
█
```

```
Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp8
```

```
❖ > java Master 9000
```

```
Master listening on 9000
```

```
Worker registered: 10.134.43.63:10001
```

```
Worker registered: 10.134.43.63:10002
```

```
█
```

```
Welcome to Hallownest, little ghost...
```

```
Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp8
```

```
• > java JobClient localhost 9000 .\input.txt WordCountMapper WordCountReducer 4 output.txt  
Job finished: {output=output.txt, message=job finished}
```

```
Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp8
```

```
❖ > █
```

EXPLORER

EXP8

exp

Collector.class

Collector.java

input.txt

JobClient.class

JobClient.java

KV.class

Reducer.java

MRRequest.java

MRResponse.java

input.txt

1

hello hello, this is mark lopes speaking

2

implementing exp 8 of mapreduce

3

hadoop hadoop

4

very nice and fun

EXPLORER

EXP8

exp

Collector.class

Collector.java

input.txt

JobClient.class

JobClient.java

KV.class

KV.java

Mapper.class

Mapper.java

Master.class

Master.java

Master\$1.class

MRRequest.class

MRRequest.java

MRRequest\$Type.class

MRResponse.class

MRResponse.java

output.txt

Reducer.class

MRRequest.java

MRResponse.java

output.txt > data

1

very 1

2

implementing 1

3

this 1

4

is 1

5

nice 1

6

speaking 1

7

mapreduce 1

8

and 1

9

of 1

10

8 1

11

hadoop 2

12

hello 2

13

exp 1

14

lopes 1

15

fun 1

16

mark 1

17